COMP1786 — Mobile Application Development

UI/UX Design Specification

Admin App (Native Android / Kotlin) · User App (React Native / Hybrid)

Standard / Basic Design vs. Modern / Professional Design

University Coursework Reference Document

1. "Current State" Analysis — The Basic Amateur Implementation

Before defining the high-quality design, it is useful to understand what a minimum-effort, basic implementation typically looks like. This section describes the "floor" against which the modern design is compared.

1.1 Visual & Structural Problems

A basic implementation of these two apps would share a recognisable set of shortcomings across both native and hybrid platforms:

- Flat, grey activity layouts with no visual hierarchy — every screen looks identical regardless of context.
- Default Android EditText widgets with no validation states, helper text, or error messages.
- No status colour coding — Active, Pending, and Completed projects are distinguished only by a plain text string.
- No confirmation screen before saving; the user taps "Save" and data is committed immediately with no opportunity to review.
- Search implemented as a full-screen separate Activity/Fragment rather than an inline, real-time filter.
- Expense list rendered as a basic ListView with no grouping, no totals, and no swipe actions.
- Hardcoded text sizes with no typographic scale; body text and headings are visually similar.
- Navigation consists of a stack of Activities (Admin) or a single screen at a time (Hybrid), with no bottom navigation or drawer.
- No empty states — when a list is empty, the user sees a blank white screen with no guidance.
- Forms use Toast messages for errors, which disappear before the user can act on them.

1.2 Basic Implementation — Screen-by-Screen Summary

Screen	Basic Behaviour
Project List (Admin)	Plain ListView of project names, one tap to open Edit Activity.
Add/Edit Project	Vertical stack of EditText fields, single Save button, no field validation.
Confirmation Screen	Not implemented — data saved directly on Save press.
Expense List	Separate Activity with a basic adapter; no grouping by type or status.
Add Expense	Simple form with Spinners for Type/Method/Status; no budget guard.
User – Project List	FlatList of project titles; no search bar, no status badges.
User – Favourites	Separate tab listing project IDs only; no persistent storage between sessions.

Screen	Basic Behaviour
User – Search	Full-screen SearchView, full page reload on every keystroke.

💡 *The basic approach scores in a Pass–Merit range. It fulfils functional requirements but demonstrates little awareness of UX principles, Material Design guidelines, or professional development standards.*

2. Modern / Professional Design — Material 3 + Modern Hybrid

2.1 Information Architecture

The Information Architecture (IA) organises screens into logical groups so that users can form an accurate mental model of the app. Both apps follow a three-tier IA:

Admin App — Navigation Structure

- Bottom Navigation Bar with three top-level destinations: Projects, Expenses, and Settings.
- Projects Destination → Project List Screen → Project Detail / Edit Screen → Confirmation Screen → (save to SQLite).
- Expenses Destination → Expense List Screen (scoped to selected project) → Add / Edit Expense Screen → Confirmation Screen.
- A FAB (Floating Action Button) on all list screens opens a "New Item" bottom sheet, reducing navigation depth.
- Search is surfaced via a persistent SearchBar at the top of both list screens — no separate search Activity.

User App — Navigation Structure

- Bottom Tab Navigator with three tabs: Browse, Favourites, and Profile.
- Browse Tab → Project List → Project Detail (read-only) — no expense creation.
- Favourites Tab → Persisted list of bookmarked projects (AsyncStorage / MMKV) with one-tap removal.
- Search is an inline SearchBar component in the Browse tab header; results filter in real time as the user types.

Screen Grouping Rationale

Grouping by task type (manage vs. view) rather than by data model (projects vs. expenses) reduces cognitive load. The Admin user needs to switch frequently between projects and their associated expenses; the Bottom Nav makes this a single tap rather than a back-stack journey.

2.2 Visual Language

Colour Palette — Semantic Status System

Colour is the single most powerful visual tool for communicating state at a glance. The palette below is derived from Material 3 tonal surface principles and provides WCAG AA contrast on all text/background pairs.

Status / Role	Chip / Badge Colour (Hex)	Usage Context
ACTIVE	Text: #1B5E20 BG: #E8F5E9	Project status chip, filter badge
PENDING	Text: #E65100 BG: #FFF3E0	Project/Expense awaiting approval

Status / Role	Chip / Badge Colour (Hex)	Usage Context
COMPLETED	Text: #01579B BG: #E1F5FE	Finished project, approved expense
PAID	Text: #4A148C BG: #F3E5F5	Expense payment processed
REJECTED	Text: #B71C1C BG: #FFEBEE	Expense claim denied

Typography Scale (Material 3 Type Tokens)

Material 3 defines named type roles rather than raw sizes. The table below maps each role to a concrete pixel equivalent for implementation.

Token	Size (sp)	Weight	Use In App
displaySmall	36	Regular	App name on splash / onboarding
headlineMedium	28	SemiBold	Screen titles (Project Detail heading)
titleLarge	22	Medium	Card titles, project names in list
titleMedium	16	Medium	Section headers inside a detail screen
bodyLarge	16	Regular	Primary body copy, field values
bodyMedium	14	Regular	Secondary info, helper text under fields
labelSmall	11	Medium	Status chips, timestamps, metadata

Spacing & Elevation System

- Base unit: 8 dp. All padding, margin, and gap values are multiples of 8 dp (8, 16, 24, 32).
- Card elevation: 1 dp (resting) → 3 dp (pressed) → 8 dp (modal bottom sheet).
- Corner radius: Cards and chips use 12 dp; dialogs and sheets use 28 dp (Material 3 large shape).
- List item height: 72 dp minimum for two-line items; 56 dp for single-line items.

2.3 Detailed Layout — Project Detail & Expense List Screen

This is the most complex screen in the Admin App and a key scoring area. It combines read/edit access to a project with an inline scrollable list of its associated expenses, totals, and quick actions.

Screen: ProjectDetailActivity (Admin App — Native Kotlin)

Layout type: CoordinatorLayout with CollapsingToolbarLayout + NestedScrollView.

▲ CollapsingToolbarLayout (height 200 dp, collapses to 56 dp)
Project Name [titleLarge, white] | ACTIVE chip | ⋮ overflow menu
← Back icon | Edit icon (pencil) | Delete icon (bin)

■ **Budget Progress Row (sticky, 56 dp)**
LinearProgressIndicator (spent / budget) | £ spent of £ budget | % remaining chip

Project Details Card (elevation 1 dp, radius 12 dp)
Row: Project ID [labelSmall, muted] · Start Date · End Date
Row: Owner [bodyMedium] · Status chip · Budget [bodyLarge, bold]
Row: Description [bodyMedium, 2-line truncated, tap to expand]

Expenses Section Header (sticky)
"Expenses (12)" [titleMedium] | Filter chips: ALL / PENDING / PAID / REJECTED | + Add FAB

[Expense Item — 72 dp — swipe right to edit, swipe left to delete]
● Claimant avatar/initials | Claimant Name [titleMedium] | PAID chip [labelSmall, purple]
Type: Travel · Method: Card · Date: 12 Mar 2025 | Amount: £240.00 [bodyLarge, right-aligned]

● Claimant avatar/initials | Claimant Name [titleMedium] | PENDING chip [labelSmall, orange]
Type: Accommodation · Method: Cash · Date: 08 Mar 2025 | Amount: £95.50 [bodyLarge, right-aligned]

▼ **Expense Summary Footer (pinned)**
Total Claimed: £1,540.00 | Approved: £1,200.00 | Pending: £340.00 | Budget Remaining: £2,460.00

💡 The CollapsingToolbar creates a dramatic entry into the screen. As the user scrolls down, the large project header collapses to a compact toolbar, keeping actions always visible while maximising list space.

Field-Level Layout for Add/Edit Project Form

The form uses Material 3 OutlinedTextField components with persistent helper text. Fields are grouped using visual dividers and section labels to reduce the perception of form length.

- Section "Identity": Project ID (read-only after creation, chip-style display), Project Name (required, max 80 chars, live char count).
- Section "Timeline": Start Date + End Date in a side-by-side DatePickerDialog row; date range validation prevents end before start with an inline error.
- Section "Ownership": Owner Name text field with auto-complete from a pre-populated list of team members.
- Section "Finance": Budget field with currency prefix icon; ExposedDropdownMenu for Status (shows colour preview chip next to each option).
- Section "Description": Multi-line field, max 500 chars, with a 4-line visible height and remaining character counter.
- Bottom: Two buttons — "Cancel" (Text style) and "Review & Save" (Filled style, primary colour) — sticky at the bottom of the scroll.

3. UX Improvements — Micro-interactions, Validation & Navigation

3.1 Micro-interactions

Micro-interactions are small, purposeful animations and feedback responses that make an interface feel alive and trustworthy. They are not decorative; each one communicates a state change.

Interaction Trigger	Micro-interaction Response
Tap Add FAB	FAB morphs into a bottom sheet with a shared-element expand animation (300 ms, EaseOutExpo curve).
Project saved successfully	Check-mark lottie animation plays on the list item for 600 ms, then item slides into sorted position.
Budget exceeded	Budget progress bar turns red with a shake animation (3 cycles, 150 ms each); field border turns red.
Swipe to delete expense	Red delete tray slides in from the right; holding > 50 % of width triggers a "release to delete" haptic pulse.
Toggle favourite (User App)	Bookmark icon performs a spring scale-up (1.0 → 1.3 → 1.0) with a colour fill transition from outline to filled.
Pull-to-refresh on list	Custom lottie spinner (branded, circular) plays during refresh; fades out on completion.
Status chip tap (filter)	Chip grows slightly (scale 1.05), border animates from dashed to solid, list filters with a fade crossfade.
Form field focus	Outline border animates from 1 dp → 2 dp; label floats up with a 150 ms colour transition to primary blue.

3.2 Inline Validation Feedback

Validation must be contextual, non-blocking, and immediate. The following strategy eliminates reliance on Toast messages, which disappear before users can act.

- Real-time validation (on text change): Character count, field-level format errors (e.g., invalid date format). The error appears under the field in red helper text — never as a Toast.
- On-focus validation (when field gains focus): Helper text changes to a contextual hint, e.g., "Enter the owner's full name as it appears in HR records."
- On-blur validation (when field loses focus): Full validation runs. If invalid, field outline turns red (Error colour token), helper text becomes an error message. A warning icon appears inside the field.
- On-submit validation: All invalid fields are highlighted simultaneously; the screen scrolls to the first error. The "Review & Save" button shows a loading spinner, then either progresses to the Confirmation Screen or returns with errors highlighted.
- Confirmation Screen pattern: The review screen shows all field values as read-only chips/text, with a summary card. Any field flagged as a warning (not an error) is shown with an amber exclamation chip. Two actions: "Back to Edit" and "Confirm & Save" (with a final haptic pulse on the latter).

3.3 Navigation Patterns

Navigation decisions have a direct impact on task completion rates. The following patterns are chosen for their alignment with Material 3 guidelines and their suitability for the app's task structure.

- Admin App: NavigationBar (bottom) with three destinations. The back stack within each destination is managed independently, so switching tabs does not lose state. Modal flows (Add/Edit + Confirmation) use a NavGraph nested graph, ensuring Back always returns correctly.
- User App (React Native): Bottom Tab Navigator (React Navigation 6) with screen options disabling the header. Each tab has its own Stack Navigator. The Favourites tab uses a PersistQueryClient pattern to survive app backgrounding.
- Search pattern: SearchBar component embedded in the screen's AppBar (both apps). The query drives a StateFlow (Admin) or useMemo-filtered array (User). No navigation event is triggered — the list filters in-place.
- Empty state screens: When a search yields no results, an illustrated empty state (SVG asset + descriptive text + CTA button) replaces the list. This prevents user confusion and provides a path forward.
- Confirmation Screen navigation: This is a required feature per the coursework spec. It is implemented as a separate Fragment/Screen (not a Dialog) to give the layout breathing room and satisfy the "confirmation before write to SQLite" requirement. The user cannot circumvent it by pressing Back from the Confirmation Screen — Back returns to the form, not to the list, preserving the mandatory review step.

4. Key Comparison Table — Basic vs. High First-Class Approach

This table provides a direct, feature-by-feature comparison suitable for use in a coursework design document. It demonstrates awareness of professional standards and positions the implemented design clearly in the upper mark band.

Feature / Criterion	Basic Approach (Pass–Merit)	High First-Class Approach (80–100%)
Search Functionality	Full-screen SearchActivity. New DB query on each button press. No filter chips. Empty result shows blank screen.	Inline SearchBar in AppBar. Real-time filtering via StateFlow/debounce (300 ms). Filter chips for Name / Description / Date / Owner. Illustrated empty state with "Clear search" CTA.
Data Entry (Project Form)	Plain vertical stack of EditText. Error shows as Toast. No helper text. No field grouping. Budget is a free-text field.	OutlinedTextField with persistent helper text. Grouped sections (Identity / Timeline / Finance). Inline error on blur. DatePickerDialog for date fields. ExposedDropdownMenu with colour chips for Status. Sticky Review & Save CTA.
Confirmation Screen (Mandatory)	Not implemented or shown as a simple AlertDialog. Back press bypasses review. Data written directly to SQLite.	Dedicated Fragment in NavGraph. All fields shown as read-only summary cards with colour-coded status. Warnings highlighted in amber. "Back to Edit" preserves form state. "Confirm & Save" writes to SQLite with haptic feedback and success animation.
Status / State Communication	Plain text string "Active" / "Pending". No colour differentiation. User must read every item to understand state.	Semantic colour chips (Active=Green, Pending=Orange, Completed=Blue, Paid=Purple, Rejected=Red). WCAG AA contrast. Visual scanning requires zero reading — state understood at a glance.
Expense List Layout	Flat ListView. No grouping by type or status. No total row. Edit requires full-screen navigation. No budget context visible.	RecyclerView with ConcatAdapter (header card + list). Filter chips by status. Swipe-to-edit (green tray) and swipe-to-delete (red tray). Budget progress bar at top. Pinned totals footer. Empty state for no expenses.
Favourites (User App)	Boolean flag in memory only. Lost on app restart. No toggle animation. Listed by project ID only.	Persisted via AsyncStorage (MMKV for performance). Spring bookmark animation on toggle. Dedicated Favourites tab with "Remove" swipe action. Count badge on tab icon. Empty state with "Browse Projects" CTA.
Navigation Architecture	Stack of Activities (Android). Each back press pops the stack. No tab navigation. Deep routes require 4+ back presses to return to home.	Single Activity + Navigation Component (Admin). Nested NavGraphs for modal flows. Bottom NavigationBar for top-level destinations. State preserved per tab. Max 2 back presses to return home from any screen.

Feature / Criterion	Basic Approach (Pass–Merit)	High First-Class Approach (80–100%)
Accessibility	No content descriptions on icons. Status communicated by colour alone (fails WCAG 1.4.1). Touch targets may be < 48 dp.	contentDescription on all icon buttons. Status communicated by colour AND text/icon (WCAG 1.4.1 pass). All touch targets ≥ 48 × 48 dp. Supports TalkBack announcement for list item state changes.

5. Implementation Checklist

Use this checklist to ensure all design decisions are implemented before submission.

Admin App (Native Android / Kotlin)

- Single Activity architecture with Navigation Component and nested NavGraph for modal flows.
- Bottom NavigationBar with Projects, Expenses, and Settings tabs.
- Project list: RecyclerView with CardView items, inline SearchBar, status chip filtering.
- Add/Edit Project: OutlinedTextField, DatePickerDialog, ExposedDropdownMenu for Status, grouped form sections.
- Confirmation Screen: Dedicated Fragment showing all field values; "Back to Edit" and "Confirm & Save" actions.
- Project Detail: CollapsingToolbarLayout + NestedScrollView, budget LinearProgressIndicator, pinned expense totals footer.
- Expense items: 72 dp two-line list items, status chips, swipe actions via ItemTouchHelper.
- Semantic status colours applied consistently across all chips and progress indicators.
- All icon buttons have contentDescription. Touch targets ≥ 48 × 48 dp.

User App (Hybrid / React Native)

- Bottom Tab Navigator with Browse, Favourites, and Profile tabs.
- Browse: FlatList with real-time search (useMemo filtered), status badge chips, pull-to-refresh.
- Project Detail: Read-only ScrollView with all project fields and expense count summary (no edit access).
- Favourites: AsyncStorage persistence, spring bookmark animation, count badge on tab icon, empty state screen.
- Search: Inline SearchBar in Browse header with debounced query (300 ms) and clear button.
- Consistent status colour tokens applied via a shared theme/constants file.
- Empty states on Browse and Favourites tabs with illustrated SVG and CTA button.